Instituto Tecnológico de Costa Rica

Escuela de Administración de Tecnologías de Información



Reporte de Laboratorio #4

Actas municipales en un grafo (Neo4j y GraphRAG)

Bases de Datos Avanzadas

Profesor:

Manuel Zumbado Corrales

Por:

Emmanuel Esquivel Chavarría - 2022312336

Yerik Chaves Serrano - 2022437493

Jose Pablo Guerrero Duarte - 2022099311

Noviembre, 2026

1 Implementación de los TODO y verificaciones

Repositorio: `https://github.com/TheKrigerenJp/Laboratorio4_BDA`. Notebook ejecutado el 9/11/2026 a las 19:07 en Docker (Neo4j 5.26 Community, un nodo); evidencia en `labs/lab4-neo4j/evidencia/`.

1.1 TODO 1 — `Persona` y `PARTICIPA_EN` (paso 5.3)

```
MATCH (a:Acta {id: $id})
MERGE (p:Persona {nombre: $nombre})
MERGE (p)-[r:PARTICIPA_EN]->(a)
SET r.rol = $rol, r.cargo = $cargo
```

```
OK: cargar dos veces no duplica nada: (18, 71) → (18, 71)
OK: (personas, relaciones) = (18, 71); se esperaba (18, 71)
OK: el cargo vive en la relación: presidencia de 61-2026 = ['Gilberth González González']
```

1.2 TODO 2 — `Fragmento` y `CONTIENE` (paso 5.4)

```
MATCH (a:Acta {id: $acta})
MERGE (f:Fragmento {id: $id})
SET f.indice = $indice, f.texto = $texto, f.embedding = $embedding
MERGE (a)-[:CONTIENE]->(f)
```

```
OK: cargar dos veces no duplica fragmentos ni relaciones
OK: conteos {'fragmentos': 15, 'contiene': 15, 'sigue': 11}; se esperaba {'fragmentos': 15,
    'contiene': 15, 'sigue': 11}
OK: todos los fragmentos tienen su vector
```

1.3 TODO 3 — Un salto (paso 6.1)

```
MATCH (a:Acta)-[:CONTIENE]->(f:Fragmento)
RETURN a.id AS acta, a.tipo AS tipo, a.fecha AS fecha, count(f) AS
    fragmentos
ORDER BY acta
```

```
{'acta': '58-2026', 'tipo': 'Ordinaria', 'fecha': '2026-09-29', 'fragmentos': 4}
{'acta': '59-2026', 'tipo': 'Extraordinaria', 'fecha': '2026-10-01', 'fragmentos': 3}
{'acta': '60-2026', 'tipo': 'Ordinaria', 'fecha': '2026-10-06', 'fragmentos': 4}
{'acta': '61-2026', 'tipo': 'Ordinaria', 'fecha': '2026-10-13', 'fragmentos': 4}
OK: una fila por acta, con las columnas acta, tipo, fecha y fragmentos
```

1.4 TODO 4 — Dos saltos (paso 6.2)

```
MATCH (p:Persona {nombre: $nombre})-[:PARTICIPA_EN]->(a:Acta)-[:CONTIENE
    ]->(f:Fragmento)
WHERE toLower(f.texto) CONTAINS toLower($palabra)
RETURN a.id AS acta, f.id AS fragmento
ORDER BY acta, f.indice
```

```
Ana Betty Valenciano Moscoso + «pésame»: ['61-2026#1', '61-2026#2']
OK: dos saltos correctos para Ana Betty Valenciano Moscoso y «pésame»
Zeneida Chaves Fernández + «aprobación»: ['58-2026#0', '58-2026#1', '60-2026#0', '60-2026#1'
    , '61-2026#0', '61-2026#1']
OK: dos saltos correctos para Zeneida Chaves Fernández y «aprobación»
```

1.5 TODO 5 — Recuperación por vector y por camino (paso 7)

Va después de `CALL db.index.vector.queryNodes(...) YIELD node, score`:

```
MATCH (a:Acta)-[:CONTIENE]->(node)
WHERE size($ids) = 0 OR a.id IN $ids
OPTIONAL MATCH (node)-[:SIGUE]->(sig:Fragmento)
OPTIONAL MATCH (p:Persona)-[r:PARTICIPA_EN]->(a)
WITH node, score, a, sig,
     collect({nombre: p.nombre, rol: r.rol, cargo: r.cargo}) AS
        participantes
RETURN node.id AS fragmento, node.texto AS texto, sig.texto AS siguiente,
       a.id AS acta, a.tipo AS tipo, a.fecha AS fecha, a.url AS url,
       score, participantes
ORDER BY score DESC
LIMIT $k
```

```
Solo vector:    [('60-2026', 0.819), ('58-2026', 0.814), ('61-2026', 0.81)]
Vector + acta:  [('61-2026', 0.81), ('61-2026', 0.646), ('61-2026', 0.636)]
OK: la consulta devuelve exactamente las nueve columnas pedidas
OK: con $k = 3 se devuelven tres fragmentos
OK: con el filtro, todos los fragmentos son del acta 61-2026
OK: los fragmentos vienen ordenados por score, de mayor a menor
OK: los participantes traen su cargo desde la relación
OK: al menos un fragmento trae el texto que le SIGUE
OK: sin un acta en la pregunta, la búsqueda recorre toda la base
```

2 Delimitación: índice y actas cargadas (pregunta 1)

```
El índice publica 4234 documentos. El corpus usa 4.
   58-2026: el PDF tiene 114 páginas; se cargan 4213 caracteres
   59-2026: el PDF tiene 26 páginas; se cargan 3481 caracteres
   60-2026: el PDF tiene 137 páginas; se cargan 3977 caracteres
   61-2026: el PDF tiene 72 páginas; se cargan 4223 caracteres
LlamaIndex: 4 documentos → 15 fragmentos
```

El índice del Concejo Municipal de Belén publica **4234 documentos**. Al grafo entraron **4 actas** (`58-2026` a `61-2026`), y de cada una solo las **2 primeras páginas**: 15 894 caracteres y 15 fragmentos, de un total de 349 páginas que suman los cuatro PDF. Ahí están el encabezado, los participantes y el orden del día. El corte es deliberado: muchas actas antiguas son PDF escaneados o enlaces que devuelven HTML, y cargar más de cuatro mil documentos no cabe en una sesión.

¿Y si solo contamos con un PDF escaneado? `pypdf` no extrae texto de una imagen, así que el paso 2 fallaría en la verificación «tiene texto extraíble» (mínimo 400 caracteres). Habría que agregar una etapa de **OCR** antes de partir el texto (por ejemplo, `ocrmypdf` o Tesseract en español), revisar la calidad del texto reconocido (tildes, nombres propios, columnas) y marcar esos fragmentos (por ejemplo, `origen: 'ocr'`) para tratarlos con cautela. El resto del flujo (fragmentos, vectores, grafo) no cambia.

3 Modelo del grafo y la propiedad `cargo` (pregunta 2)

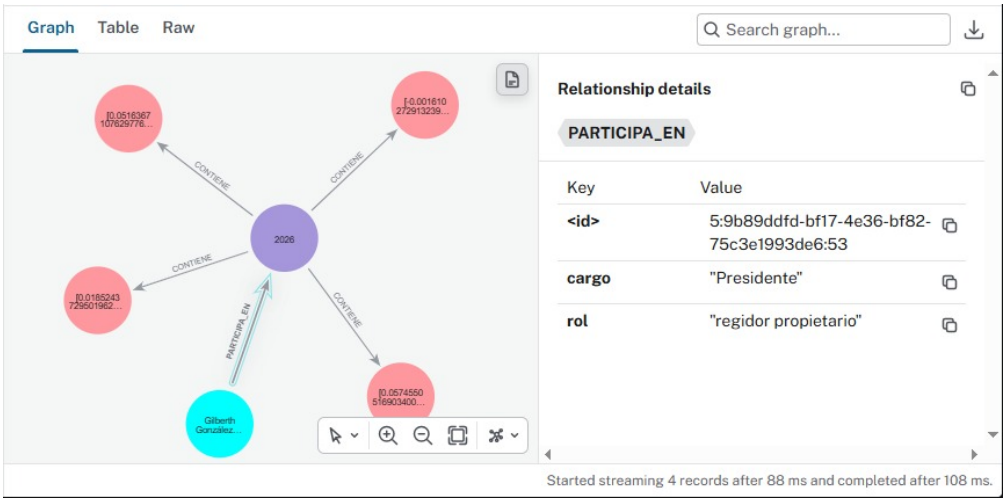


Figura 1: Navegador de Neo4j, paso 6.3: camino `Persona`→`Acta`→`Fragmento` del acta 61-2026; el panel muestra `cargo` en la relación `PARTICIPA_EN`.

`cargo` es una propiedad de `PARTICIPA_EN` y no de `Persona` porque **describe la participación de una persona en una sesión concreta, no a la persona**. El grafo tiene 18 personas pero 71 participaciones: cada persona aparece en varias actas, y el cargo puede cambiar entre sesiones. Como `Persona` se identifica solo por `nombre`, guardar el cargo en el nodo obligaría a sobrescribirlo en cada carga: la última acta cargada «ganaría» y se perdería la historia. En la relación, cada par persona–acta guarda su propio `rol` y `cargo`, igual que en SQL iría en la tabla intermedia `participacion` y no en `persona`.

4 Consultas de uno y dos saltos frente a SQL (pregunta 3)

Las consultas y sus resultados están en las secciones 1.3 y 1.4. El TODO 3 en SQL es un `JOIN` (`acta`–`fragmento`) más un `GROUP BY`. En Cypher el `JOIN` es la flecha `-[:CONTIENE]->` y la agrupación es implícita. El TODO 4 en SQL usa **cuatro tablas y tres `JOIN`** (`persona`–`participacion`–`act` porque la relación muchos-a-muchos necesita una tabla intermedia. En Cypher es un solo patrón de camino.

Un tercer salto (por ejemplo, al fragmento siguiente por `SIGUE`) agrega al menos un `JOIN` más: un auto-`JOIN` de `fragmento`, o dos si `sigue` se modela como tabla aparte. Se pasa de 3 a **4 o 5 `JOIN`**, mientras que en Cypher basta con agregar `-[:SIGUE]->(g:Fragmento)` al patrón. Además, Neo4j sigue punteros desde el nodo de partida (*index-free adjacency*): el costo crece con los vecinos visitados, no con el tamaño de las tablas.

5 `CREATE` en lugar de `MERGE` en el TODO 1 (pregunta 4)

- **`CREATE` para la persona.** Las 18 personas aparecen en varias actas (71 participaciones). Ya en la **primera** carga, el `CREATE` de un nombre repetido viola la restricción `persona_nombre`, y Neo4j lanza `ConstraintError`. La celda lo convierte en «VERIFICACIÓN FALLÓ: la carga intentó crear de nuevo un nodo que ya existía». Sin la restricción quedarían 71 nodos `Persona` en vez de 18 (142 tras la segunda carga), y las consultas por nombre devolverían varios nodos.
- **`CREATE` para la relación** (con `MERGE` en la persona). No hay restricción sobre relaciones, así que no da error, pero la segunda carga duplica cada `PARTICIPA_EN`: de `(18, 71)` se pasaría a `(18, 142)` y fallaría la verificación «cargar dos veces no duplica nada».

`MERGE` hace la carga **idempotente**: repetirla deja el mismo grafo, `(18, 71)` → `(18, 71)`.

6 «Solo vector» frente a «vector + acta» (pregunta 5)

```
Tokens del codificador: ['_¿', 'Quién', '_pre', 'side', '_la', '_sesión', '_61', '-20', '26'
    , '?']
El codificador lee como máximo 128 tokens por texto.
Los fragmentos tienen entre 91 y 375 tokens: 12 de 15 superan el límite.
   fragmento   coseno   (1 + coseno) / 2
   60-2026#0   0.638    0.819
   58-2026#0   0.627    0.814
   61-2026#0   0.620    0.810
```

Con **solo vector** (salida del TODO 5, sección 1.5), el fragmento más parecido a «¿Quién preside la sesión 61-2026?» es `60-2026#0`, seguido de `58-2026#0`, y el de `61-2026` queda **tercero**. Los tres *scores* están separados por menos de 0.01. Los tres son el fragmento 0 de su acta, el encabezado («Acta de la Sesión Ordinaria N° …, celebrada por el Concejo

Municipal de Belén…» más la lista de participantes), casi idéntico en todas las actas. La búsqueda vectorial encuentra textos **parecidos**, no **correctos**. El número del acta, lo único que cambia, se parte en tokens genéricos (`_61`, `-20`, `26`) que casi no mueven el vector. Con **vector + acta**, los tres fragmentos son de `61-2026`: `CONTIENE` dice con exactitud a qué acta pertenece cada fragmento, y el filtro `a.id IN $ids` descarta el resto.

Límite de 128 tokens. El codificador solo lee los primeros 128 tokens de cada fragmento, y **12 de los 15 lo superan** (hasta 375). Lo que viene después del token 128 no influye en el vector. En el fragmento 0 de cada acta, esos 128 tokens son casi todo encabezado y nombres de participantes, idénticos entre sesiones. El contenido que distingue una sesión de otra queda fuera de lo que el modelo lee. Por eso los *scores* quedan tan juntos (0.819 / 0.814 / 0.810) y el orden entre actas es prácticamente arbitrario.

7 Lectura crítica de las respuestas del modelo (pregunta 6)

```
P1: Según el acta 61-2026, ¿quién preside la sesión?
   contexto 61-2026#0 score=0.816 · 61-2026#1 score=0.695 · 61-2026#3 score=0.653
   Respuesta: Gilberth González González es preside la sesión Ordinaria del Concejo
       Municipal de Belén, según la acta 61-2026.
P2: En el acta 61-2026, ¿qué acta anterior se somete a aprobación?
   contexto 61-2026#0 score=0.770 · 61-2026#1 score=0.697 · 61-2026#3 score=0.675
   Respuesta: El acta 61-2026 se somete a aprobación al acta 61-2026.
P3: ¿En qué sesión se acordó enviar una nota de pésame?
   contexto 60-2026#3 score=0.750 · 58-2026#0 score=0.705 · 61-2026#0 score=0.698
   Respuesta: La nota de pésame fue enviada en la sesión ordinaria del 2026-10-13, cuando se
       celebraba la sesión extraordinaria.
OBSERVACIÓN: la primera respuesta sí nombra a quien el grafo registra como Presidente.
   TODO 1: OK   TODO 2: OK   TODO 3: OK   TODO 4: OK   TODO 5: OK
OK: el laboratorio está completo.
```

#	¿Correcta?	Análisis
P1	**Sí** (error gramatical)	Los «Hechos del grafo» dicen `Presidente: Gilberth González González` y el fragmento `61-2026#0` lo confirma. El modelo usó el hecho estructural que viene de recorrer `PARTICIPA_EN` en el TODO 5.
P2	**No: alucinación**	El contexto sí traía la respuesta: `61-2026#1` dice «somete a aprobación el Acta de la Sesión Ordinaria **N°60-2026**». El modelo respondió «el acta **61-2026**», que es el acta de la pregunta, no la anterior. La verificación automática no lo detecta porque `61-2026` sí aparece en el contexto: solo revisa que no se citen actas ajenas, no que la cita sea correcta.
P3	**No sustentada: alucinación**	Ninguno de los tres fragmentos recuperados habla de un pésame: `60-2026#3` trata de un oficio que menciona la «Sesión Extraordinaria No.50-2026», y los otros dos son encabezados. Los acuerdos de pésame están en `61-2026#1` y `#2` (los encontró el TODO 4 por palabra clave), pero el vector no los recuperó. La fecha «2026-10-13» coincide por casualidad con la sesión correcta, pero salió del encabezado de `61-2026#0`. «Cuando se celebraba la sesión extraordinaria» mezcla texto de `60-2026#3`. Lo correcto era decir que la información no aparece en el contexto.

Cuadro 1: Evaluación de las tres respuestas del paso 9 contra el contexto recuperado.

Criterio. Una respuesta es correcta si lo que afirma aparece en los fragmentos o en los «Hechos del grafo» que recibió el generador (paso 8.5). Es alucinación todo dato afirmado sin apoyo en ese contexto, o en contra de él. El generador es pequeño (Qwen2.5-0.5B, 494 millones de parámetros). Acierta cuando el dato viene como hecho explícito del grafo (P1). Falla cuando debe leer y razonar sobre el texto (P2) o cuando la recuperación vectorial no trae el fragmento correcto (P3). Esta última combina los dos límites de la sección 6: sin acta en la pregunta no hay filtro estructural, y el vector se queda con encabezados parecidos. Una búsqueda híbrida (palabra clave + vector) habría recuperado `61-2026#1` y `#2`.